

Universidad Abierta Interamericana

Trabajo Final

Sistema de Gestión de Ventas

TechStore S.A.

Análisis Técnico

Alumnos

Ruffinengo Yasmin

Noya Ana

Civilotti Genaro

Julio de 2026

Índice

1. Propósito y alcance	2
2. Resumen de la solución	2
2.1. Vista general de la arquitectura	3
3. Decisiones arquitectónicas	3
3.1. Arquitectura en capas	3
3.2. MVC adaptado a una aplicación de escritorio	4
4. Patrones implementados	4
4.1. Modelo–Vista–Controlador	4
4.2. Singleton	4
4.3. Data Mapper y ORM	5
5. Estrategia de integración entre componentes	5
5.1. Integración interna	5
5.2. Integración con SQL Server	5
5.3. Integración con el servicio de facturación	6
5.4. Integración de reportes	7
6. Diagrama de clases	8

1. Propósito y alcance

Este documento describe y justifica la arquitectura implementada para el Sistema de Gestión de Ventas de TechStore S.A. El análisis se basa en la solución desarrollada y cubre:

- la arquitectura general y la aplicación del patrón MVC;
- los patrones de diseño utilizados y los recomendados para evolucionar el sistema;
- la estrategia de integración entre la interfaz, la lógica de negocio, la persistencia, SQL Server y el servicio externo de facturación;
- el tratamiento de transacciones, validaciones, errores, configuración y seguridad;
- la trazabilidad de los requerimientos;
- los riesgos técnicos, las limitaciones actuales y las mejoras propuestas;
- la estrategia de pruebas y despliegue.

La solución es una aplicación de escritorio para Windows desarrollada con .NET y Windows Forms. La persistencia se realiza en SQL Server mediante Entity Framework Core con un enfoque Code First.

2. Resumen de la solución

La solución está organizada en cuatro proyectos:

Cuadro 1: Proyectos y responsabilidades

Proyecto	Rol arquitectónico	Responsabilidad principal
Vista	Vista	Formularios, grillas, captura de datos, navegación y presentación de mensajes.
Controladora	Controlador / aplicación	Validaciones, coordinación de casos de uso, cálculos y reglas de negocio.
Modelo	Acceso a datos e integración	Repositorios, consultas LINQ, transacciones, contexto de EF Core, migraciones y carga de configuración.
Entidades	Modelo de dominio y contratos	Entidades, relaciones, enumeraciones y DTO utilizados por reportes y consultas.

2.1. Vista general de la arquitectura

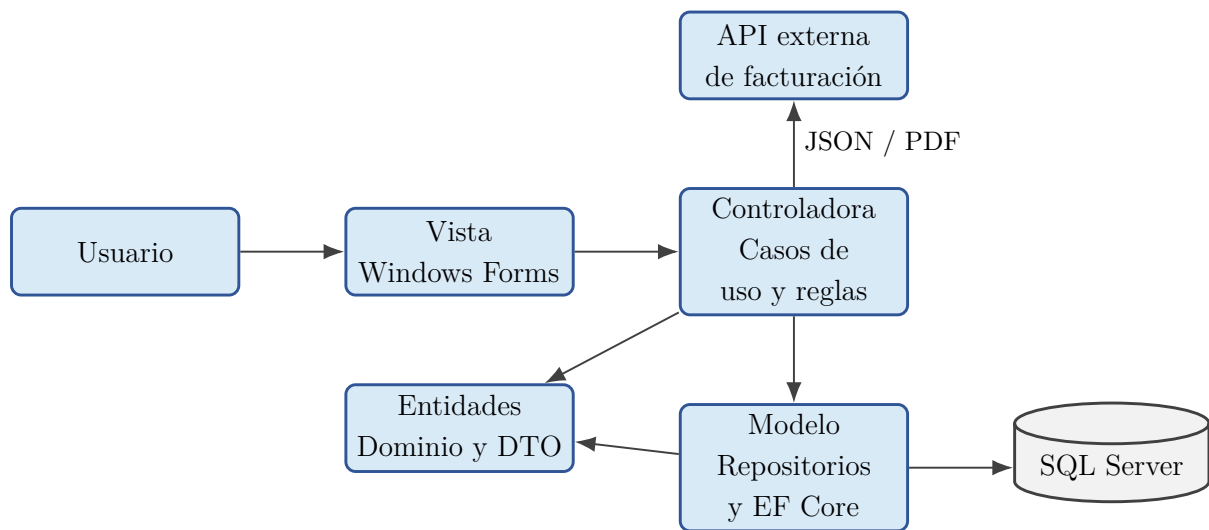


Figura 1: Arquitectura general del sistema

En la implementación actual, **Vista** también referencia **Modelo** para cargar el archivo de configuración y comprobar la conexión durante el inicio. Esta dependencia funciona, pero sería conveniente encapsularla en un servicio de inicialización para mantener más estrictamente el límite entre capas.

3. Decisiones arquitectónicas

3.1. Arquitectura en capas

La separación en proyectos reduce el acoplamiento entre interfaz, reglas y persistencia. Una modificación visual no debería cambiar las consultas a la base de datos; del mismo modo, un cambio de persistencia debería afectar principalmente al proyecto **Modelo**.

Las dependencias deben apuntar hacia el dominio:

1. **Vista** conoce a **Controladora** y **Entidades**.
2. **Controladora** conoce a **Modelo** y **Entidades**.
3. **Modelo** conoce a **Entidades** y a Entity Framework Core.
4. **Entidades** no debería depender de la interfaz ni de la base de datos.

Esto favorece la mantenibilidad solicitada, aunque todavía existe margen para mejorar mediante interfaces e inyección de dependencias.

3.2. MVC adaptado a una aplicación de escritorio

El patrón MVC se aplica de la siguiente manera:

Cuadro 2: Correspondencia entre MVC y la solución

Elemento MVC	Implementación	Ejemplos
Modelo	Entidades del dominio y repositorios de persistencia.	Venta, Producto, Inventario, RepositorioVenta.
Vista	Formularios Windows Forms.	frmVenta, frmProducto, frmReporteVenta.
Controlador	Clases que reciben las solicitudes de la Vista y coordinan reglas y repositorios.	ControladoraVenta, ControladoraProducto, ControladoraReporte.

Por ejemplo, la vista de ventas captura la selección del cliente, la sucursal, el vendedor, el método de pago y los productos, y construye una entidad **Venta**. Después, la controladora correspondiente valida y calcula los importes y delega la persistencia al repositorio de ventas.

4. Patrones implementados

En la solución se incluyen patrones arquitectónicos y de persistencia que resuelven problemas comunes y cuya aplicación nos otorga diversos beneficios

4.1. Modelo–Vista–Controlador

Problema resuelto. Evitar que presentación, reglas de negocio y acceso a datos queden mezclados en los formularios.

Aplicación. Los formularios representan la Vista; las entidades y repositorios forman el Modelo; y las controladoras coordinan las operaciones.

Beneficio. Permite modificar la interfaz o las reglas con menor impacto sobre el resto del sistema y asignar responsabilidades claras a los integrantes.

4.2. Singleton

Problema resuelto. Brindar un punto de acceso único a cada controladora desde los distintos formularios.

Aplicación. Las controladoras exponen una propiedad estática **Instancia** y mantienen su constructor privado.

Beneficio. Simplifica el acceso y evita crear repetidamente objetos coordinadores.

Costo. Introduce estado global y dependencias ocultas, dificulta las pruebas aisladas y la ejecución concurrente. Es aceptable para el alcance académico y el proceso único de escritorio, pero para una evolución productiva se recomienda sustituirlo por inyección de dependencias con un ciclo de vida controlado.

4.3. Data Mapper y ORM

Problema resuelto. Traducir entre objetos C# y tablas relacionales.

Aplicación. Entity Framework Core mapea entidades, claves, relaciones e índices mediante `DbContext` y Fluent API.

Beneficio. Permite expresar consultas con LINQ, administrar relaciones y versionar el esquema mediante migraciones.

5. Estrategia de integración entre componentes

5.1. Integración interna

La integración entre los proyectos es directa, en memoria y sin serialización:

1. La Vista captura y valida datos de formato básico.
2. La Vista crea entidades o parámetros de consulta.
3. La Controladora valida reglas de negocio y coordina el caso de uso.
4. El Repositorio traduce la operación a consultas y comandos de EF Core.
5. EF Core ejecuta SQL parametrizado sobre SQL Server.
6. El resultado retorna como entidad, colección de sólo lectura, DTO o mensaje de operación.

Las entidades se utilizan en operaciones transaccionales y los DTO en consultas preparadas para presentación.

5.2. Integración con SQL Server

`Context` deriva de `DbContext` y expone un `DbSet` por agregado persistente. La cadena de conexión se obtiene desde `DB_DSN`; no se encuentra escrita en el código.

Las consultas de lectura usan mayormente `AsNoTracking`, lo que reduce el costo del seguimiento cuando el resultado no va a modificarse. Para la venta, el contexto sí mantiene seguimiento del inventario porque debe detectar y persistir la disminución del stock.

Las migraciones permiten:

- crear el esquema inicial;
- reproducir relaciones, índices y datos iniciales;
- incorporar de forma versionada la cuenta corriente;
- mantener alineadas las instalaciones de todos los integrantes.

5.3. Integración con el servicio de facturación

La generación del comprobante sigue este flujo:

1. El usuario selecciona una venta ya registrada.
2. La Vista solicita a **ControladoraVenta** la venta completa con cliente, método de pago, detalle y productos.
3. **ControladoraFactura** transforma la venta en el contrato JSON externo.
4. Envía una solicitud **POST** autenticada con Bearer token.
5. Solicita y valida una respuesta **application/pdf**.
6. Retorna los bytes del PDF a la Vista.
7. La Vista guarda el archivo en la ubicación elegida por el usuario.

La URL y la clave se leen desde **FACTURACION_API_URL** y **FACTURACION_API_KEY**.

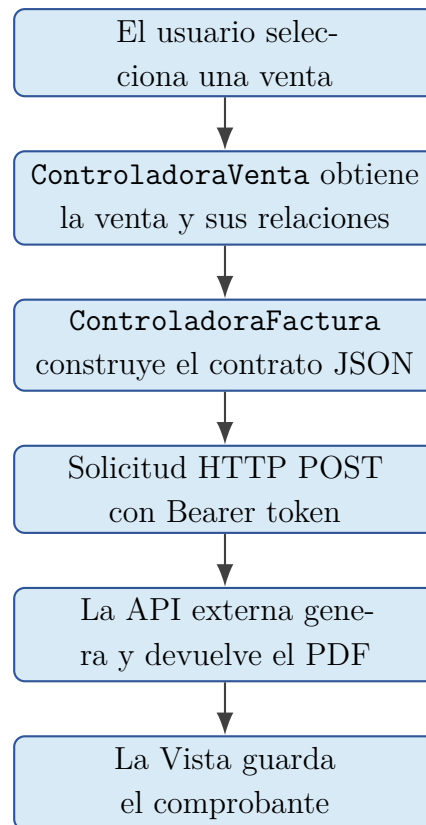


Figura 2: Flujo de integración con el servicio de facturación

La llamada es asíncrona para no bloquear la interfaz. Se reutiliza una instancia estática de `HttpClient`, evitando el agotamiento de conexiones.

5.4. Integración de reportes

Los filtros se reciben en la controladora y se componen en el repositorio mediante LINQ. Entity Framework traduce agrupaciones, sumas, filtros y proyecciones a SQL para que el procesamiento ocurra en la base de datos.

Para incluir el día final completo se utiliza un intervalo semiabierto:

Listing 1: Tratamiento del rango de fechas

```
FechaVenta >= fechaDesde.Date  
FechaVenta < fechaHasta.Date.AddDays(1)
```

Esta forma evita problemas producidos por componentes de hora, minutos y segundos.

6. Diagrama de clases

El siguiente diagrama representa las entidades persistentes del sistema, sus atributos principales, claves primarias y foráneas, enumeraciones, clases asociativas y multiplicidades. Los DTO utilizados por los reportes no se incluyen porque son proyecciones de consulta y no forman parte del modelo de dominio persistente.

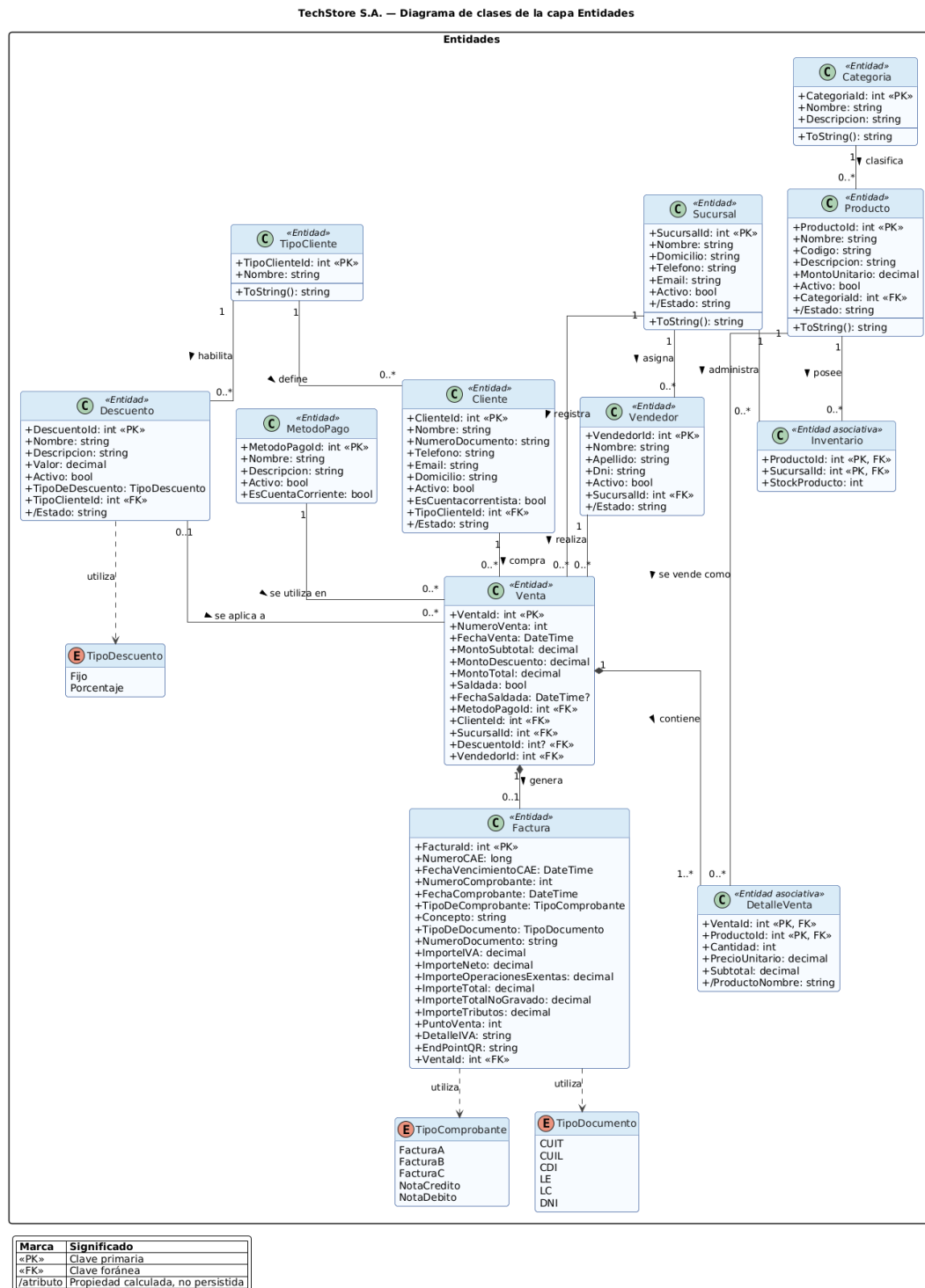


Figura 3: Diagrama de clases de la capa Entidades